

Distributed Reservation Management System

SOEN 423 — Assignment 1 Design Documentation

1. System Overview

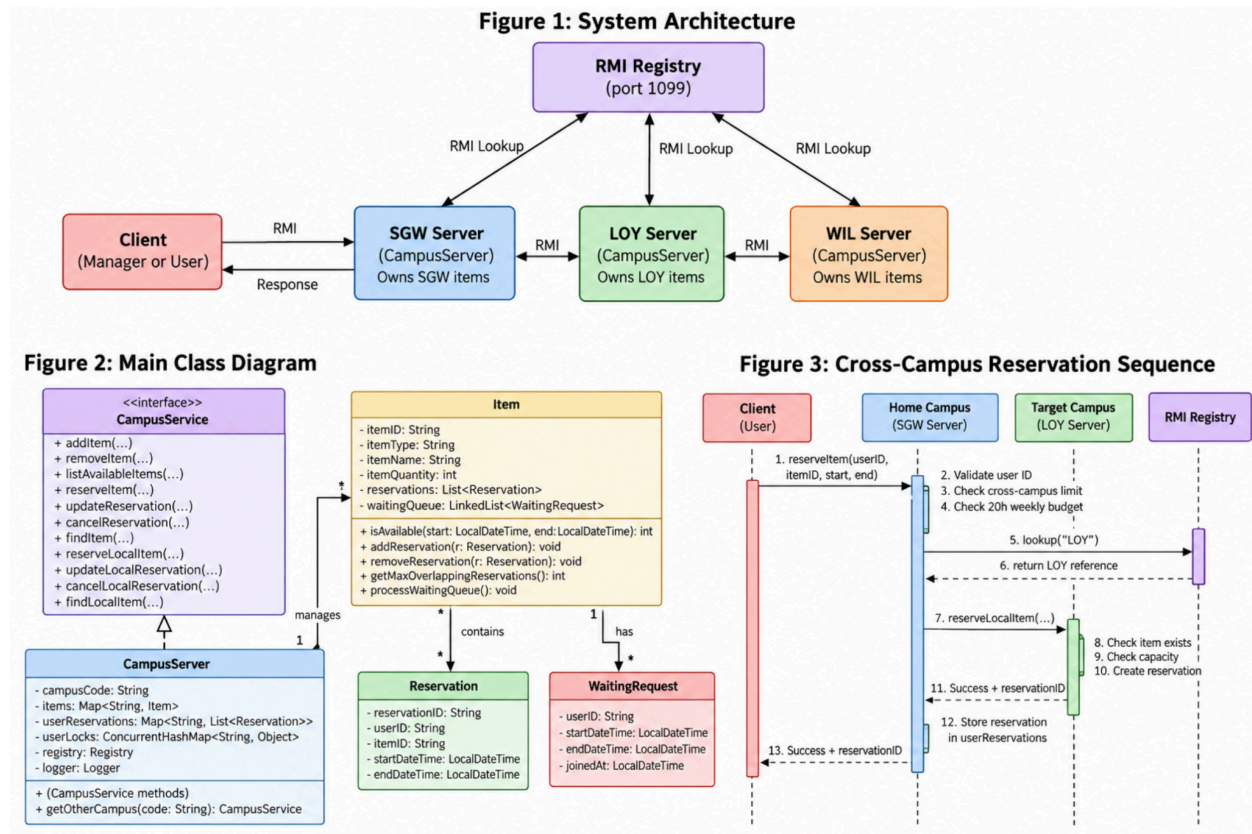
The Distributed Reservation Management System (DRMS) manages reservable items across three Concordia campuses: Sir George Williams (SGW), Loyola (LOY), and West Island (WIL). Each campus is represented by its own `CampusServer` remote object. Clients communicate with their home campus using Java RMI, while campus servers may communicate with one another through the same RMI interface for cross-campus operations.

The user's or manager's home campus is determined from the first three characters of their ID. For example, `SGWU1111` is an SGW user and initially communicates with the SGW server. This follows the assignment requirement that all operations first go through the actor's home campus.

The current implementation creates the three `CampusServer` objects inside `ServerMain` and registers them as `SGW`, `LOY`, and `WIL` in an RMI registry running on port 1099. Therefore, the campuses are separate remotely accessible server objects, although the current demonstration configuration launches them inside the same JVM.

2. Architecture

The system follows a client-server architecture with the home campus acting as the coordinator for user operations.



The client first validates the ID and performs a lookup of the corresponding home campus in the RMI registry. Manager and user menus then invoke operations through the **CampusService** remote interface.

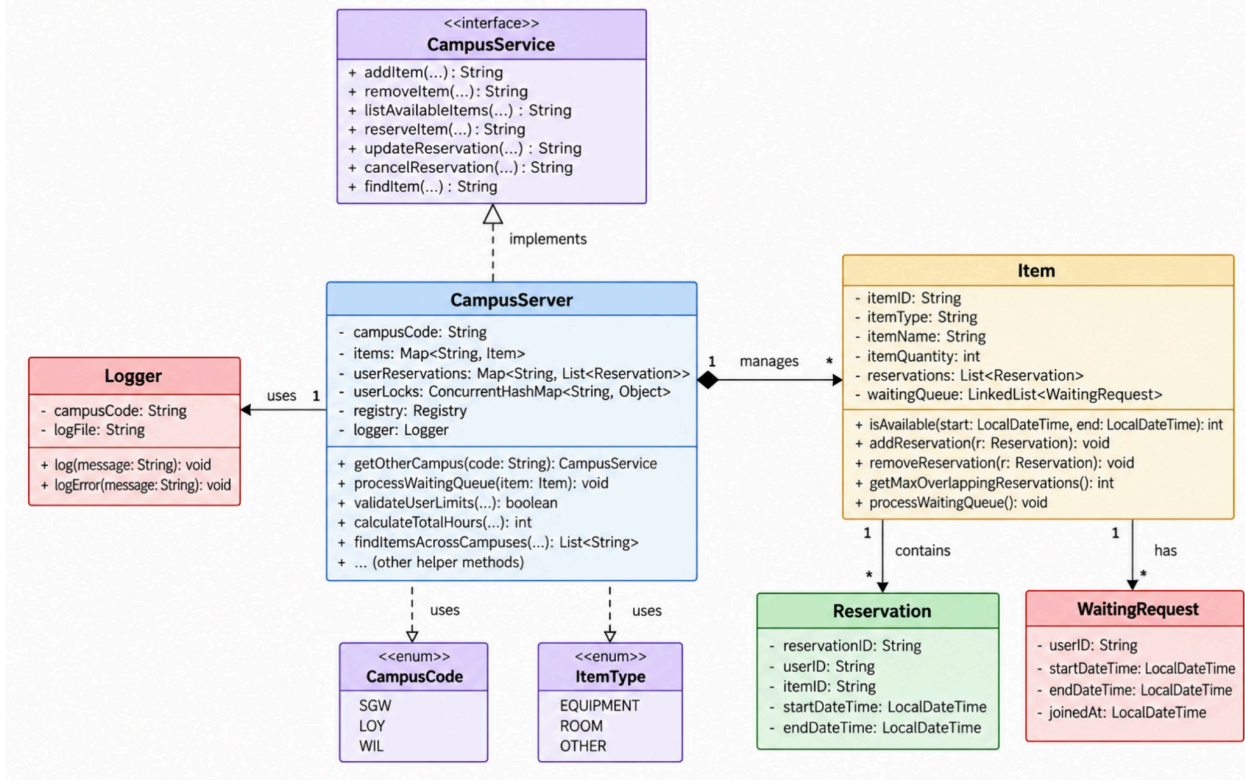
Each campus server has two main responsibilities. First, it owns and manages the items whose IDs start with its campus code. Second, when it is the user's home campus, it coordinates user-level rules such as the weekly reservation budget and cross-campus reservation limit.

For example, if **SGWU1111** reserves **LOY1001**, the request is first received by SGW. SGW verifies the user's weekly budget and cross-campus restrictions and then obtains the LOY RMI reference. LOY performs the capacity check and creates the actual reservation. If LOY succeeds, SGW stores a corresponding reservation in its **userReservations** structure so that future budget and cross-campus checks can be performed locally.

3. Main Classes and Data Structures

The main classes are:

Figure 2: Main Class Diagram



Item storage

Each **CampusServer** uses a **HashMap<String, Item>** where the key is the item ID. This gives direct lookup of an item without scanning every item on the server.

Each **Item** contains an **ArrayList<Reservation>** containing its reservations and a **LinkedList<WaitingRequest>** used as the item's FIFO waiting queue.

User reservation storage

The home campus maintains:

ConcurrentHashMap<String, List<Reservation>> userReservations

The user ID is the key and the value contains that user's reservations. This structure is used to calculate the weekly 20-hour budget and determine whether another overlapping reservation is allowed at a non-home campus.

Per-user locks

A **ConcurrentHashMap<String, Object>** stores one synchronization object for each user. Operations such as reservation, update, cancellation, and joining a waiting queue synchronize on the corresponding user's lock.

This is important because the weekly budget and cross-campus restrictions depend on the user's complete reservation state. Two concurrent requests from the same user must not both pass validation before either reservation has been recorded.

Capacity calculation

Capacity is calculated using a sweep-line technique. For reservations overlapping the requested interval, the implementation inserts **+1** at reservation start times and **-1** at end times into a `TreeMap<LocalDateTime, Integer>`. Traversing these events determines the maximum number of simultaneously active reservations.

Because reservation intervals are treated as half-open intervals, a reservation ending at 12:00 does not conflict with another starting exactly at 12:00.

Weekly budget calculation

Reservations are split according to calendar weeks. `getHoursByWeek()` finds the Monday corresponding to each portion of the reservation and calculates the number of hours belonging to that week.

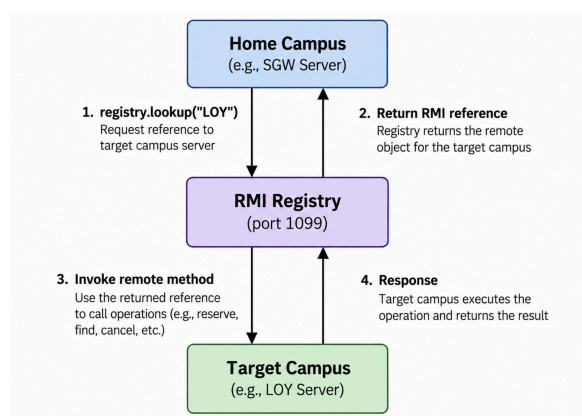
The home server then compares the requested hours with the hours already used by the user. This also correctly handles reservations that cross a Monday boundary. The assignment allows at most 20 reservation hours per calendar week.

4. Communication Between Servers

All client-to-server and server-to-server communication uses Java RMI.

`CampusService` extends `Remote` and exposes both client operations such as `reserveItem`, `findItem`, and `cancelReservation`, and internal server operations such as `reserveLocalItem`, `findLocalItem`, and `cancelLocalReservation`.

A server obtains another campus through the shared RMI registry:



For `findItem`, the home server searches itself and sends requests to the other two campuses. Remote searches are executed with a finite two-second timeout. If a campus cannot be reached, the result explicitly reports that campus as `UNAVAILABLE` rather than treating it as having no matching items.

5. Synchronization and Consistency

Concurrency is handled at two levels.

Server-level synchronized operations protect item state. Methods that modify local item reservations or quantities use synchronization so that two threads cannot simultaneously modify the same server state in an unsafe manner.

User-level locks protect rules that apply across multiple campuses. A reservation operation obtains the user's lock at the home server before evaluating the cross-campus restriction, weekly budget, and performing the reservation.

An important design decision is to avoid holding the campus server monitor while making unnecessary remote calls. Holding multiple server locks while servers call each other could create distributed deadlock. The implementation therefore uses the user's lock for home-campus coordination and local server synchronization for capacity changes.

For cross-campus reservations, the target campus is modified first. The home-campus reservation record is added only after the target campus reports success. Updates follow a similar strategy: the target reservation is changed first, and the home-campus copy is changed only if the remote operation succeeds. Cancellation removes the reservation from the item-owning campus before removing the home server's tracking copy.

This provides simple commit-style consistency without requiring a full distributed transaction protocol. A limitation is that the implementation does not provide durable recovery for a network failure occurring after a remote campus commits but before the home server receives the response. For the scope of this assignment, state is maintained in memory and server logs provide an audit trail.

6. Waiting Queue

Each item owns one FIFO waiting queue, implemented with `LinkedList<WaitingRequest>`. A waiting request stores the user, requested interval, and time at which it joined the queue.

The queue is offered only after the user's cross-campus and weekly-budget checks have succeeded but the item does not currently have sufficient capacity.

When capacity may have changed, such as after a cancellation, update, or quantity increase, waiting queues are processed again.

The server examines requests in FIFO order. A request whose interval has expired is removed. If capacity is still unavailable it remains in the queue. When capacity exists, the item-owning campus contacts the user's home campus. The home campus re-evaluates the weekly budget and cross-campus rule before

performing the reservation. A request is removed from the queue only after the reservation succeeds. This allows temporarily ineligible requests to remain in their original queue position as required by the specification.

7. Logging

Each campus maintains a separate text log such as `SGW_server.log`. Logging is synchronized so concurrent operations do not write conflicting log entries.

Every log entry contains the timestamp, processing server, operation, actor ID, and operation result. Both successful and rejected requests are therefore recorded.

8. Test Scenarios

The following scenarios test both normal operation and important edge/failure cases.

Test	Scenario	Expected Result / Purpose
T1	SGW manager creates <code>SGW1001</code> with quantity 2 and lists items.	Item is created and displayed with correct current availability. Tests basic manager operations.
T2	SGW manager attempts to create <code>LOY1001</code> .	Request is rejected because managers may modify only their home-campus items.
T3	Create quantity 1 item. Reserve it from 11:00–12:00 and reserve it again from 12:00–13:00.	Both requests succeed. Tests the half-open interval rule.
T4	Create quantity 1 item. Two users request overlapping intervals.	First succeeds and second reports insufficient capacity. Tests capacity enforcement.
T5	After T4, second user joins the waiting queue. Cancel the first reservation.	Waiting request is reconsidered and automatically receives the released capacity if still eligible. Tests queue processing.
T6	SGW user reserves a LOY item, then attempts another overlapping LOY reservation.	Second request is rejected by the non-home-campus overlapping reservation limit.
T7	SGW user holds overlapping reservations at LOY and WIL.	Both are allowed because the limit is evaluated separately for each non-home campus.

T8	User accumulates exactly 20 hours of reservations in one week and attempts one additional hour.	20 hours succeeds; additional reservation fails and remaining weekly budget is reported.
T9	Make a reservation that crosses Monday midnight.	Hours before Monday count toward the first week and hours after Monday count toward the next. Tests weekly splitting.
T10	Cancel a reservation and then make another reservation using the released hours.	New reservation succeeds, showing that cancellation credits the weekly budget.
T11	Update a reservation to an interval with insufficient capacity or one that exceeds the budget.	Update fails and original reservation remains unchanged. Tests atomic update behavior.
T12	Call <code>findItem</code> when matching items exist at SGW, LOY, and WIL, including one with zero free units.	All matching items are returned with interval-specific availability, including zero-capacity items.
T13	Make one campus unavailable during <code>findItem</code> .	Search still returns other campuses and explicitly reports the unavailable campus instead of reporting zero items.
T14	Launch two concurrent reservations for the same quantity-1 item/time interval.	Only one succeeds. Tests synchronization and prevention of overbooking.
T15	Use a manager ID for a user operation or a user ID for a manager operation.	Request is rejected. Tests role validation.
T16	Attempt to remove an item containing a current/future reservation.	Removal is rejected and item remains unchanged.
T17	Attempt to reduce item quantity below its maximum overlapping reservation count.	Update is rejected. Tests preservation of existing reservations.
T18	Perform successful and rejected requests and inspect the campus log.	Both outcomes appear with timestamp, server, actor, operation, and result.

These tests cover the major functional requirements as well as concurrency, distributed communication, failure handling, consistency, interval boundaries, waiting queues, and validation. The assignment specifically requires the demonstration to include both normal and failure/concurrency cases.

9. Most Important / Difficult Part

The most difficult part of the assignment was maintaining consistent state during **cross-campus operations under concurrency**.

A reservation is not determined only by whether an item has capacity. The system must also enforce the user's weekly 20-hour budget and ensure that the user does not have two overlapping reservations at the same non-home campus. These rules are maintained by the home campus, while actual item capacity is maintained by the item-owning campus.

This creates a distributed synchronization problem. For example, without synchronization, two simultaneous requests from the same SGW user could both check the user's current LOY reservations, both see that no conflict exists, and then both reserve separate LOY items. The resulting state would violate the cross-campus restriction.

The implementation addresses this using a **per-user lock on the home campus**. The lock serializes reservation-related operations belonging to the same user while still allowing unrelated users to execute concurrently. After the home-campus checks succeed, the target server performs its capacity modification using synchronized local operations.

Waiting queues make this problem more difficult because a queued request may no longer be valid when capacity eventually becomes available. Therefore, queue assignment is not performed solely by the item server. The item server contacts the user's home campus again, where the budget and cross-campus constraints are re-evaluated before the reservation is committed.

This combination of user-level synchronization, item-level synchronization, RMI communication, and revalidation was the most important design challenge in the system.

10. Conclusion

The DRMS separates item ownership from user-level coordination. Each campus maintains its own items and reservation capacity, while a user's home campus maintains enough reservation information to enforce global user restrictions.

Java RMI provides communication between clients and campuses as well as communication among campus servers. Hash maps provide efficient item and user lookup, lists store reservation histories, FIFO queues manage waiting requests, and synchronization protects shared state from concurrent requests.

The resulting design supports local and cross-campus reservations, capacity management, weekly budgets, waiting queues, updates, cancellations, distributed searches, failure reporting, and concurrent requests while keeping the responsibilities of each campus clearly separated.